Modelling Physical Climate Risk in IFRS 9 ECL Frameworks

A Plain-Language Guide to the ECL Prototype

Two-Regime Markov-Switching Approach to Credit Migration — Zimbabwe's Microfinance Sector

---

Harare Institute of Technology (HIT) — Department of Applied Mathematics — March 2026

Contents

What Is This Project?

The Problem in Plain English

Microfinance lenders in Zimbabwe give small loans to people who farm or run informal businesses. When a drought comes, borrowers struggle to repay — their crops fail, their income falls, and they miss payments. Under the international accounting rule IFRS 9, lenders must set aside money *before* losses happen (a "provision"). The amount set aside is called the **Expected Credit Loss (ECL)**.

The usual way of calculating ECL ignores whether it is drought season or not. It uses one fixed set of probabilities for borrowers moving from one credit rating to another. This project builds a smarter model that has *two sets* of probabilities — one for normal weather and one for drought — and switches between them depending on a drought index.

Why It Matters

- If ECL is underestimated during a drought, a lender could run out of capital and collapse.
- If ECL is overestimated every quarter, the lender holds back too much money and cannot lend to the next borrower.
- A climate-aware model gives a fairer, more accurate number.

The Answer: A Two-Regime Switching Model

Think of it like a thermostat that switches between two modes:

Condition	Mode	Which matrix is used?
Drought ($\gamma < -1.0$)	Stress	M_{stress} (higher default risk)
Normal ($\gamma \geq -1.0$)	Normal	M_{normal} (lower default risk)

In reality the model uses a smooth blend of both matrices, not a hard switch, so there is no abrupt jump at the threshold. The blending formula is:

$$P(\gamma) \;=\; \omega(\gamma) \cdot M_{\text{stress}} \;+\; \big(1 - \omega(\gamma)\big) \cdot M_{\text{normal}}$$

where $\omega(\gamma)$ is a number between 0 and 1 that gets closer to 1 the deeper the drought is.

The Dataset

File Location

```
prototype/data/jay_dataset.csv
```

What Each Column Means

Column	Meaning
`borrower_id`	Unique identifier for each borrower (e.g. `MF01234`).
`loan_id`	Unique identifier for the loan.
`rating`	Credit rating at that point in time. 1 = default (worst), 5 = best.
`obs_date`	The date this observation was recorded (one record per quarter per borrower).
`district`	The district the borrower lives in.
`gamma`	The SPEI drought index for that period. Negative means dry; positive means wet. Below -1.0 is considered a drought.
`default_flag`	1 if the borrower is in default this quarter, 0 otherwise.
`loan_amount`	Outstanding loan balance in US dollars.

Key Numbers in the Dataset

- **6 borrowers**, observed over roughly 10 quarters each.
- **59 total observations**, producing **51 valid transitions** (pairs of consecutive quarterly ratings for the same borrower).
- **31 normal-regime** and **20 stress-regime** transitions.
- **5 credit states**: 1 (default/absorbing), 2, 3, 4, 5.
- Drought index γ ranges from about -1.84 to $+0.84$.

What is SPEI? The Standardised Precipitation Evapotranspiration Index (SPEI) measures how dry or wet conditions are compared with the long-term average. A value of 0 is normal; -1 means moderately dry; -2 means severely dry; $+1$ means unusually wet.

How the System Works — Stage by Stage

The prototype is built from five Python modules, each responsible for one stage of the pipeline.

$$\text{CSV data} \longrightarrow \underbrace{\text{Stage 1}}_{\texttt{ingestion}} \longrightarrow \underbrace{\text{Stage 2}}_{\texttt{transitions}} \longrightarrow \underbrace{\text{Stage 3}}_{\texttt{estimation}} \longrightarrow \underbrace{\text{Stage 4}}_{\texttt{forecasting}} \longrightarrow \underbrace{\text{Stage 5}}_{\texttt{validation}}$$

Stage 1 — Data Loading (`src/ingestion.py`)

What it does in plain English: Reads the CSV file from disk, checks that all required columns are present, parses dates, makes sure ratings are between 1 and 5, and sorts everything in chronological order. If anything is wrong (file not found, missing column, bad rating value) it raises an error immediately before any calculation starts.

Key functions:

- `load_dataset(path)` — opens the file, parses and cleans it.
- `validate_dataset(df)` — checks columns and rating range.
- `dataset_summary(df)` — returns a dictionary of headline statistics (number of borrowers, date range, etc.).

Stage 2 — Transition Extraction (`src/transitions.py`)

What it does in plain English: For each borrower, looks at pairs of consecutive quarterly records and records what rating they moved from and to. For example, if a borrower was rated 3 in March and then rated 2 in June, that is a "3 → 2" transition. It then labels each transition as *normal* or *stress* depending on the SPEI value at that time.

Important rule: Transitions *from* State 1 (default) are excluded. Once a borrower defaults, they stay in default — there is nothing to model.

Key functions:

- `extract_rating_transitions(df)` — produces the transitions table.
- `build_count_matrix(transitions, regime)` — counts how many times each $i \to j$ transition was observed. Can be filtered to just normal or just stress transitions.
- `mle_matrix_from_counts(counts)` — converts counts into probabilities. If a rating state was never observed (e.g. State 5 had no transitions), a sensible "prior" probability is used so no row is left as all zeros.

Example count matrix (normal regime):

$$C_{\text{normal}} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 5 & 0 & 0 & 0 \\ 0 & 0 & 8 & 3 & 0 \\ 0 & 0 & 1 & 6 & 2 \\ 0 & 0 & 0 & 1 & 2 \end{pmatrix}$$

Row i, column j = number of times a borrower moved from State i to State j during normal conditions.

Stage 3 — Model Estimation (`src/estimation.py`)

What it does in plain English: Estimates the two matrices (M_{normal} and M_{stress}) and the two blending parameters (κ and γ_0) from the data.

Two-stage approach (why not do everything at once?) With only 51 transitions and 34 parameters, fitting everything simultaneously is unstable — the computer can reach wild numbers like $\kappa = 9 \times 10^{117}$. Instead:

1. **Stage 3a — Matrix estimation:** Use the hard-threshold labels (normal/stress) to compute M_{normal} and M_{stress} directly from the count matrices.
2. **Stage 3b — Scalar optimisation:** With the matrices fixed, find the values of κ and γ_0 that make the observed transitions most probable (maximum likelihood over just two numbers).

What κ and γ_0 mean:

- γ_0 is the "tipping point" — the SPEI level at which the model considers conditions to have switched from normal to drought. Typically around -0.9.
- κ controls how sharply the switch happens. A large κ means almost a hard switch; a small κ means a very gradual blend.

The regime weight formula:

$$\omega(\gamma) = \frac{1}{1 + e^{\kappa(\gamma - \gamma_0)}}$$

This is a decreasing S-curve: when $\gamma \ll \gamma_0$ (deep drought), $\omega \to 1$ (full stress weight); when $\gamma \gg \gamma_0$ (wet), $\omega \to 0$ (full normal weight).

Testing the two-regime model (LRT): The Likelihood Ratio Test asks: "does adding the second regime actually help?" It compares the log-likelihood of the two-regime model to a simpler single-regime model. A small p-value (below 0.05) would mean the two-regime model is significantly better. With only 51 transitions the test lacks power, but the structural distance metric (SVDM) still shows meaningful separation.

SVDM (Singular Value Decomposition Metric): A number that measures how different M_{normal} and M_{stress} are from each other. An SVDM of 0 means identical matrices (no regime effect); a large SVDM means the two regimes behave very differently.

Stage 4 — ECL Forecasting (`src/forecasting.py`)

What it does in plain English: For each active borrower, projects how likely they are to default over the next 5 years (20 quarters) and discounts those losses back to today.

Step by step:

1. **Pick the blended matrix** for the current climate (γ): $P(\gamma) = \omega \cdot M_{\text{stress}} + (1 - \omega) \cdot M_{\text{normal}}$.
2. **Multiply the matrix by itself** h times (P^h) to get the probability of being in each state after h quarters.
3. **Read off the default column** to get the cumulative probability of having defaulted by quarter h.

4. **Calculate the marginal (period-by-period) default probability**: how much did the default probability increase in quarter h compared to quarter $h-1$?
5. **Apply the ECL formula** to each quarter and sum:

$$\text{ECL} = \sum_{h=1}^{20} \underbrace{(1+r)^{-h/4}}_{\text{discount factor}} \times \underbrace{\text{EAD}}_{\text{loan balance}} \times \underbrace{\text{LGD}}_{\text{loss fraction}} \times \underbrace{\text{PD}_h}_{\text{marginal default prob}}$$

where $r = 5\%$ per year and LGD $= 45\%$ (standard assumption).

Scenario analysis: The same calculation is repeated four times, using different fixed values of γ to represent different climate futures:

Scenario	γ override	Probability weight
Baseline	-0.73	55%
Moderate Drought	-1.20	25%
Severe Drought	-1.80	12%
Wet / Recovery	$+0.80$	8%

The probability-weighted average across all four scenarios gives the *expected ECL* that IFRS 9 requires.

Stage 5 — Backtesting (`src/validation.py`)

What it does in plain English: Checks how well the model predicted what actually happened, using the same data it was trained on (in-sample check, because the dataset is too small for a true out-of-sample test).

Metrics:

- **AUC (Area Under the ROC Curve):** Measures discrimination — how well does the model separate borrowers who defaulted from those who did not?
  - 0.5 = no better than random.
  - 1.0 = perfect.
  - Values above 0.70 are considered acceptable in credit risk.
- **Brier Score:** Measures calibration — are the predicted probabilities numerically accurate? Think of it as the mean squared error of probabilities.
  - 0 = perfect.
  - Lower is better.

Both metrics are computed for the two-regime model and for a benchmark single-regime model so they can be compared side by side.

Understanding the App Screens

The Streamlit application has five pages, accessible from the left sidebar.

Page 1 — Dataset Overview

Summary cards at the top: Total observations, number of borrowers, number of stress-regime observations, and date range. These confirm the data loaded correctly before any model is run.

Rating Distribution bar chart: Shows how many observations fall into each credit state (1 to 5). A healthy portfolio should have most borrowers in states 3–5. A large bar at state 1 indicates many defaults.

SPEI Time Series: A line chart of the drought index over time for the longest-observed borrower. Red shading marks quarters where $\gamma < -1.0$ (drought / stress regime). This gives a visual sense of how often drought conditions occurred in the data period.

Raw data table: The full loaded dataset so you can inspect individual records.

Page 2 — Transition Analysis

Count matrices (one tab per regime): Each cell shows how many times a borrower moved from the row state to the column state. A cell on the diagonal means the borrower stayed in the same state. Cells in column 1 count defaults.

Probability matrices (heatmaps): The count matrices converted to row-conditional probabilities. Darker colours indicate higher probability. Comparing the Normal and Stress heatmaps visually shows how default probabilities rise under drought.

Pooled tab: All transitions combined, ignoring regime — this is the single-regime baseline.

Page 3 — Regime Model Estimation

Model parameter cards:

- κ — sharpness of the switch (higher = more abrupt).
- γ_0 — SPEI threshold value (the tipping point).
- Log-likelihood — how well the model fits the data overall (higher / less negative = better).
- Convergence status — did the optimiser finish successfully?

Side-by-side matrix heatmaps (M_{normal} vs. M_{stress}): The estimated probability matrices for each regime. Compare the default column (column 1, leftmost) across both: under stress the default probabilities should be noticeably higher.

Regime weight curve: The S-shaped logistic curve $\omega(\gamma)$ plotted over the range of drought index values. The vertical dashed line marks γ_0 (the midpoint where $\omega = 0.5$). Blue shading on the left represents drought conditions; the shaded region on the right represents normal/wet conditions.

LRT results card: Likelihood Ratio Test outcome. Shows the LR statistic, degrees of freedom, and p-value. A p-value below 0.05 would mean the two-regime model is statistically significantly better.

SVDM card: The structural distance between the two regime matrices, with a bootstrap 95% confidence interval. An SVDM of zero means the two matrices are identical.

Page 4 — ECL Scenario Simulator

Portfolio ECL table: One row per active borrower showing their current rating, loan balance, stress weight ω, one-quarter default probability, 5-year cumulative default probability, and ECL amount. The final row shows the portfolio total.

Scenario analysis table and bar chart: Each scenario's portfolio ECL, its probability weight, and weighted contribution. The bar chart shows ECL by scenario; a horizontal dashed line marks the probability-weighted expected ECL. Taller bars on the drought scenarios show the climate sensitivity.

PD fan chart: Cumulative default probability over 20 quarters for one selected rating state, plotted separately under each climate scenario. Steeper curves mean faster default accumulation.

Sliders: Allow the user to adjust LGD and the forecast horizon interactively and see ECL recalculate instantly.

Page 5 — Backtesting Metrics

Model comparison table: AUC and Brier Score for both the two-regime and single-regime models side by side. An improvement in AUC and a reduction in Brier Score confirm the two-regime model is better.

ROC curves: Two curves on the same chart — one for each model. A curve closer to the top-left corner is better (higher true-positive rate at low false-positive rate). The diagonal line represents random guessing.

Regime breakdown metrics: AUC computed separately for normal-regime and stress-regime transitions, showing whether the model performs better under one condition than the other.

Process Tests

What the Tests Are For

The tests verify that each stage of the pipeline produces correct output *before* the real dataset is touched. They use small, hand-crafted datasets so a failure points directly to the responsible function. Running the tests after any code change confirms nothing was broken.

All 82 tests run in under 3 seconds.

Test Files and What They Cover

File	What it tests
`test_ingestion.py`	File not found; missing columns; out-of-range ratings; correct types; sorted order; `dataset_summary` keys and counts.
`test_transitions.py`	Absorbing state excluded; regime labels; count matrix shape and dtype; regime filtering; row-stochastic MLE matrix; zero-count rows handled.
`test_estimation.py`	Regime weight at drought/wet/threshold; no overflow; array input; blend matrix row-stochastic; estimate_model converges with $\kappa > 0$; LRT returns finite statistic and valid p-value.
`test_forecasting.py`	Cumulative PD monotone and in $[0, 1]$; marginal PD non-negative; ECL positive and higher under drought; EAD scaling; portfolio total row; scenario count and expected row.
`test_validation.py`	AUC returns NaN for single class; perfect/-worst/random AUC; Brier score 0 for perfect; non-negative; ROC array lengths; backtest columns; y_true binary; y_pred in $[0, 1]$; validation table returns 2 rows.

How to Run the Tests

Step 1 — Open a terminal and navigate to the prototype folder:

```
cd /path/to/physical-climate-risk/prototype
```

Step 2 — Run all tests with `uv`:

```
uv run pytest tests/ -v
```

The `-v` flag shows each test name and PASSED / FAILED.

To run only one test file:

```
uv run pytest tests/test_ingestion.py -v
```

To run one specific test by name:

```
uv run pytest
   tests/test_estimation.py::TestRegimeWeight::test_deep_drought_approaches_
   -v
```

Expected output when all tests pass:

```
======================== 82 passed, 8 warnings in 1.89s
   ========================
```

Note on the `uv` tool. This project uses `uv` to manage dependencies. It is a fast Python package manager. Running any command with `uv run ...` automatically uses the correct virtual environment and installed packages. You do not need to activate a virtual environment manually.

Test Code Listings

The complete source for each test file is shown below for reference.

5.4.1 Stage 1 — tests/test_ingestion.py

```
"""
Stage 1 tests: data loading and validation.
"""
import sys, os, unittest, tempfile
import pandas as pd

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from src.ingestion import load_dataset, validate_dataset,
    dataset_summary

VALID_CSV = """\
borrower_id,loan_id,rating,obs_date,district,gamma,default_flag,loan_amount
B01,L01,3,2022-03-31,Harare,-0.50,0,5000
B01,L01,2,2022-06-30,Harare,-1.20,0,4800
B02,L02,4,2022-03-31,Bulawayo,0.30,0,8000
B02,L02,1,2022-06-30,Bulawayo,-1.50,1,0
"""

def _write_temp_csv(content):
    tmp = tempfile.NamedTemporaryFile(
        mode="w", suffix=".csv", delete=False)
    tmp.write(content); tmp.close()
    return tmp.name

class TestLoadDataset(unittest.TestCase):

    def test_file_not_found(self):
        with self.assertRaises(FileNotFoundError):
            load_dataset("/does/not/exist.csv")

    def test_valid_csv_loads(self):
        path = _write_temp_csv(VALID_CSV)
        df = load_dataset(path)
        self.assertEqual(len(df), 4)
        os.unlink(path)

    def test_sorted_by_borrower_and_date(self):
        path = _write_temp_csv(VALID_CSV)
        df = load_dataset(path)
        for _, grp in df.groupby("borrower_id"):
            dates = grp["obs_date"].tolist()
            self.assertEqual(dates, sorted(dates))
        os.unlink(path)

class TestValidateDataset(unittest.TestCase):

```

```
    def test_missing_column_raises(self):
        df = pd.DataFrame({"borrower_id": ["B01"],
                           "rating": [3]})   # missing columns
        with self.assertRaises(ValueError):
            validate_dataset(df)

    def test_out_of_range_rating_raises(self):
        path = _write_temp_csv(
            "borrower_id,loan_id,rating,obs_date,"
            "district,gamma,default_flag,loan_amount\n"
            "B01,L01,9,2022-03-31,Harare,-0.5,0,5000\n")
        df = pd.read_csv(path)
        df["obs_date"] = pd.to_datetime(df["obs_date"])
        df["rating"] = pd.to_numeric(df["rating"])
        df["gamma"] = pd.to_numeric(df["gamma"])
        df["loan_amount"] = pd.to_numeric(df["loan_amount"])
        df["default_flag"] = 0
        with self.assertRaises(ValueError):
            validate_dataset(df)
        os.unlink(path)

class TestDatasetSummary(unittest.TestCase):

    def test_summary_counts_match(self):
        path = _write_temp_csv(VALID_CSV)
        df = load_dataset(path)
        s = dataset_summary(df)
        self.assertEqual(s["Total observations"], 4)
        self.assertEqual(s["Unique borrowers"], 2)
        os.unlink(path)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Listing 1: test_ingestion.py — Stage 1: Data Loading and Validation

5.4.2 Stage 2 — tests/test_transitions.py

```
"""
Stage 2 tests: transition extraction and count matrices.
"""
import sys, os, unittest
import numpy as np, pandas as pd

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from src.transitions import (
    extract_rating_transitions, build_count_matrix,
    mle_matrix_from_counts,
)
from config import N_STATES

def _make_panel():
```

```
    rows = [
        {"borrower_id":"B01","rating":3,"obs_date":pd.Timestamp("2022-03-31"),
         "gamma":-0.5,"loan_amount":5000,"default_flag":0,
         "district":"Harare","loan_id":"L01"},
        {"borrower_id":"B01","rating":2,"obs_date":pd.Timestamp("2022-06-30"),
         "gamma":-1.5,"loan_amount":4800,"default_flag":0,
         "district":"Harare","loan_id":"L01"},
        {"borrower_id":"B01","rating":1,"obs_date":pd.Timestamp("2022-09-30"),
         "gamma":-1.8,"loan_amount":0,"default_flag":1,
         "district":"Harare","loan_id":"L01"},
        {"borrower_id":"B02","rating":4,"obs_date":pd.Timestamp("2022-03-31"),
         "gamma":0.3,"loan_amount":8000,"default_flag":0,
         "district":"Bulawayo","loan_id":"L02"},
        {"borrower_id":"B02","rating":3,"obs_date":pd.Timestamp("2022-06-30"),
         "gamma":0.2,"loan_amount":7500,"default_flag":0,
         "district":"Bulawayo","loan_id":"L02"},
    ]
    return pd.DataFrame(rows)

class TestExtractRatingTransitions(unittest.TestCase):

    def setUp(self):
        self.t = extract_rating_transitions(_make_panel())

    def test_absorbing_state_excluded(self):
        self.assertTrue((self.t["from_rating"] != 1).all())

    def test_expected_transition_count(self):
        # 3->2 (normal), 2->1 (stress), 4->3 (normal) = 3 total
        self.assertEqual(len(self.t), 3)

    def test_regime_labels_correct(self):
        normal = self.t[self.t["gamma"] >= -1.0]
        self.assertTrue((normal["regime"] == "normal").all())
        stress = self.t[self.t["gamma"] < -1.0]
        self.assertTrue((stress["regime"] == "stress").all())

class TestMleMatrixFromCounts(unittest.TestCase):

    def test_row_stochastic(self):
        counts = np.zeros((N_STATES, N_STATES), dtype=int)
        counts[2, 1] = 1; counts[2, 2] = 1
        P = mle_matrix_from_counts(counts)
        np.testing.assert_allclose(
            P.sum(axis=1), np.ones(N_STATES), atol=1e-9)

    def test_absorbing_row(self):
        counts = np.zeros((N_STATES, N_STATES), dtype=int)
        P = mle_matrix_from_counts(counts)
        expected = np.zeros(N_STATES); expected[0] = 1.0
        np.testing.assert_allclose(P[0], expected, atol=1e-12)
```

```
66
67 if __name__ == "__main__":
68     unittest.main(verbosity=2)
```

Listing 2: test_transitions.py — Stage 2: Transition Extraction

5.4.3 Stage 3 — tests/test_estimation.py

```
"""
Stage 3 tests: regime weight, model estimation, LRT.
"""
import sys, os, unittest
import numpy as np, pandas as pd

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from src.estimation import (
    regime_weight, blend_matrices,
    estimate_model, likelihood_ratio_test,
)
from config import N_STATES

def _make_transitions():
    rows = [
        {"borrower_id":"B01","from_rating":3,"to_rating":2,
         "gamma":-0.5,"regime":"normal"},
        {"borrower_id":"B01","from_rating":2,"to_rating":1,
         "gamma":-1.5,"regime":"stress"},
        {"borrower_id":"B02","from_rating":4,"to_rating":3,
         "gamma":0.3,"regime":"normal"},
        {"borrower_id":"B02","from_rating":3,"to_rating":3,
         "gamma":0.1,"regime":"normal"},
        {"borrower_id":"B03","from_rating":5,"to_rating":4,
         "gamma":-1.2,"regime":"stress"},
        {"borrower_id":"B03","from_rating":4,"to_rating":4,
         "gamma":-0.8,"regime":"normal"},
        {"borrower_id":"B04","from_rating":2,"to_rating":2,
         "gamma":-1.8,"regime":"stress"},
        {"borrower_id":"B04","from_rating":3,"to_rating":2,
         "gamma":-2.0,"regime":"stress"},
    ]
    return pd.DataFrame(rows)

class TestRegimeWeight(unittest.TestCase):

    def test_deep_drought_approaches_one(self):
        w = regime_weight(-10.0, kappa=2.0, gamma0=-1.0)
        self.assertGreater(w, 0.99)

    def test_wet_conditions_approaches_zero(self):
        w = regime_weight(10.0, kappa=2.0, gamma0=-1.0)
        self.assertLess(w, 0.01)

```

```
    def test_at_threshold_is_half(self):
        w = regime_weight(-1.0, kappa=2.0, gamma0=-1.0)
        self.assertAlmostEqual(w, 0.5, places=9)

    def test_no_overflow_extreme_gamma(self):
        w_low  = regime_weight(-100.0, kappa=5.0, gamma0=-1.0)
        w_high = regime_weight( 100.0, kappa=5.0, gamma0=-1.0)
        self.assertFalse(np.isnan(w_low) or np.isnan(w_high))

class TestEstimateModel(unittest.TestCase):

    def setUp(self):
        self.result = estimate_model(_make_transitions())

    def test_kappa_positive(self):
        self.assertGreater(self.result["kappa"], 0.0)

    def test_matrices_row_stochastic(self):
        for key in ["M_normal", "M_stress"]:
            row_sums = self.result[key].sum(axis=1)
            np.testing.assert_allclose(
                row_sums, np.ones(N_STATES), atol=1e-9)

    def test_empty_transitions_raises(self):
        with self.assertRaises(ValueError):
            estimate_model(pd.DataFrame())

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Listing 3: test_estimation.py — Stage 3: Model Estimation

5.4.4 Stage 4 — tests/test_forecasting.py

```
"""
Stage 4 tests: cumulative PD, marginal PD, ECL, scenarios.
"""
import sys, os, unittest
import numpy as np, pandas as pd

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from src.forecasting import (
    cumulative_pd, marginal_pd, compute_ecl,
    portfolio_ecl, scenario_analysis,
)
from config import N_STATES, DEFAULT_HORIZON, SCENARIOS

def _P():
    return np.array([
        [1.00,0.00,0.00,0.00,0.00],
        [0.10,0.80,0.07,0.02,0.01],
        [0.04,0.06,0.78,0.09,0.03],
```

```
        [0.01,0.02,0.06,0.82,0.09],
        [0.00,0.01,0.03,0.08,0.88],
    ])

def _Ps():  # stress (higher defaults)
    return np.array([
        [1.00,0.00,0.00,0.00,0.00],
        [0.20,0.70,0.06,0.03,0.01],
        [0.08,0.10,0.70,0.09,0.03],
        [0.03,0.04,0.08,0.76,0.09],
        [0.01,0.02,0.04,0.10,0.83],
    ])

class TestCumulativePD(unittest.TestCase):

    def test_non_decreasing(self):
        cum = cumulative_pd(3, _P(), DEFAULT_HORIZON)
        self.assertTrue(np.all(np.diff(cum) >= -1e-12))

    def test_values_in_unit_interval(self):
        cum = cumulative_pd(3, _P(), DEFAULT_HORIZON)
        self.assertTrue(np.all(cum >= 0) and np.all(cum <=
            1+1e-12))

    def test_higher_risk_borrower_higher_pd(self):
        cum2 = cumulative_pd(2, _P(), DEFAULT_HORIZON)
        cum5 = cumulative_pd(5, _P(), DEFAULT_HORIZON)
        self.assertGreater(cum2[-1], cum5[-1])

class TestComputeECL(unittest.TestCase):

    def test_ecl_positive(self):
        r = compute_ecl(3, 5000.0, -0.5, _P(), _Ps(), 2.0, -1.0)
        self.assertGreater(r["ecl"], 0.0)

    def test_ecl_higher_under_drought(self):
        ecl_n =
            compute_ecl(3,5000.0,-0.5,_P(),_Ps(),2.0,-1.0)["ecl"]
        ecl_d =
            compute_ecl(3,5000.0,-1.8,_P(),_Ps(),2.0,-1.0)["ecl"]
        self.assertGreater(ecl_d, ecl_n)

    def test_ead_scaling(self):
        e1 =
            compute_ecl(3,5000.0,-0.5,_P(),_Ps(),2.0,-1.0)["ecl"]
        e2 =
            compute_ecl(3,10000.0,-0.5,_P(),_Ps(),2.0,-1.0)["ecl"]
        self.assertAlmostEqual(e2/e1, 2.0, places=6)

class TestPortfolioECL(unittest.TestCase):

```

```
    def test_portfolio_total_row(self):
        loans = pd.DataFrame([
            {"borrower_id":"B01","rating":3,"loan_amount":5000,"gamma":-0.5}
            {"borrower_id":"B02","rating":4,"loan_amount":8000,"gamma":-0.5}
        ])
        df = portfolio_ecl(loans, _P(), _Ps(), 2.0, -1.0)
        self.assertIn("PORTFOLIO TOTAL", df["Borrower
           ID"].values)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Listing 4: test_forecasting.py — Stage 4: ECL Forecasting

5.4.5 Stage 5 — tests/test_validation.py

```
"""
Stage 5 tests: AUC, Brier Score, backtest predictions.
"""
import sys, os, unittest
import numpy as np, pandas as pd

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from src.validation import (
    calculate_auc, calculate_brier_score,
    backtest_predictions, validation_metrics,
)
from src.transitions import build_count_matrix,
   mle_matrix_from_counts

def _P(dp=0.05):
    K = 5; P = np.zeros((K,K))
    P[0,0] = 1.0
    for i in range(1,K):
        P[i,0] = dp; P[i,i] = 1-dp
    return P

def _transitions():
    rows = [
        {"borrower_id":"B01","from_rating":2,"to_rating":1,
         "gamma":-1.5,"regime":"stress"},
        {"borrower_id":"B02","from_rating":3,"to_rating":1,
         "gamma":-1.8,"regime":"stress"},
        {"borrower_id":"B03","from_rating":4,"to_rating":4,
         "gamma":-0.5,"regime":"normal"},
        {"borrower_id":"B04","from_rating":5,"to_rating":4,
         "gamma":0.3,"regime":"normal"},
        {"borrower_id":"B05","from_rating":3,"to_rating":3,
         "gamma":-0.2,"regime":"normal"},
        {"borrower_id":"B06","from_rating":2,"to_rating":2,
         "gamma":-1.2,"regime":"stress"},
    ]
```

```
    return pd.DataFrame(rows)

class TestCalculateAUC(unittest.TestCase):

    def test_single_class_returns_nan(self):
        result = calculate_auc(np.array([0,0,0]),
           np.array([0.1,0.2,0.3]))
        self.assertTrue(np.isnan(result))

    def test_perfect_returns_one(self):
        result = calculate_auc(np.array([1,1,0,0]),
                               np.array([0.9,0.8,0.1,0.2]))
        self.assertAlmostEqual(result, 1.0, places=6)

class TestCalculateBrierScore(unittest.TestCase):

    def test_perfect_returns_zero(self):
        result = calculate_brier_score(np.array([1,0]),
                                       np.array([1.0,0.0]))
        self.assertAlmostEqual(result, 0.0, places=9)

    def test_non_negative(self):
        result = calculate_brier_score(np.array([1,0,0]),
                                       np.array([0.6,0.3,0.1]))
        self.assertGreaterEqual(result, 0.0)

class TestBacktestPredictions(unittest.TestCase):

    def test_y_true_binary(self):
        df = backtest_predictions(
            _transitions(), _P(0.04), _P(0.12), 2.0, -1.0)
        self.assertTrue(set(df["y_true"].unique()).issubset({0,1}))

    def test_y_pred_in_unit_interval(self):
        df = backtest_predictions(
            _transitions(), _P(0.04), _P(0.12), 2.0, -1.0)
        self.assertTrue((df["y_pred"] >= 0).all()
                        and (df["y_pred"] <= 1).all())

class TestValidationMetrics(unittest.TestCase):

    def test_returns_two_rows(self):
        t = _transitions()
        M_single = mle_matrix_from_counts(
            build_count_matrix(t, regime=None))
        df = validation_metrics(
            t, _P(0.04), _P(0.12), 2.0, -1.0, M_single)
        self.assertEqual(len(df), 2)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Listing 5: test_validation.py — Stage 5: Backtesting Metrics

Quick Reference

Task	Command
Run all tests	`uv run pytest tests/ -v`
Run one test file	`uv run pytest tests/test_ingestion.py -v`
Start the app	`uv run streamlit run app.py`
Compile this PDF	`pdflatex explanation.tex`

Project file structure summary:

`prototype/`
`app.py` — Streamlit application (5 pages)
`config.py` — All constants and hyperparameters
`data/jay_dataset.csv` — The MFI panel dataset
`src/ingestion.py` — Stage 1: data loading
`src/transitions.py` — Stage 2: transition extraction
`src/estimation.py` — Stage 3: model estimation
`src/forecasting.py` — Stage 4: ECL calculation
`src/validation.py` — Stage 5: backtesting
`src/svdm.py` — SVDM and bootstrap confidence intervals
`src/plots.py` — All charts and figures
`tests/` — Unit tests (one file per stage)

`document/document.tex` — Full research dissertation (LaTeX)
`explanation/explanation.tex` — This document